

Full Stack Development Week 10

Topics covered:

- Full Stack Development using React, Node, Express and MySQL
- GraphQL
- React Documentation

Activity 0 (Course Experience Survey)

We value your feedback on this course! Your input helps us continuously improve the learning experience for current and future students.

The Course Experience Survey (CES) is now open. Please take a few moments to complete this anonymous survey at:

<https://surveys.rmit.edu.au/blue/>

Your thoughtful feedback is important to us - thank you for taking the time to share your perspective.

Activity 1 - Creating a Node Express MS SQL Server app with GraphQL

The backend Node, Express app will use TypeORM to create tables etc, GraphQL does not have the capability to do this.

The difference in this example and the ones covered in previous weeks is the use of GraphQL to create API for database operations instead of REST.

NOTE: if you missed lectional 9 and have no idea what GraphQL is, this lab will not make any sense. Please watch recordings of lectional 9.

Step 1:

Create a Node.js project, by now you should know how to do this

Step 2:

Install necessary libraries via the terminal/command prompt in the project:

```
npm install express @apollo/server graphql graphql-tag mssql nodemon typeorm
```

Step 3:

Copy the contents of **lectorial09/example2/backend** folder (<https://github.com/rmit-fsd-2026-s1/lectorial06-onwards/blob/main/lectorial09.zip>) into your project. It contains nothing new as compared to last week's lecture GraphQL example. All database model files / entities, creation operations are implemented using TypeORM. You should modify database credentials in `data-source.ts` to match yours.

Step 4:

Now spend time with the lab assistant to understand how the API has been developed using GraphQL syntax.

In particular, you need to understand concepts already covered in lecture 9 such as:

- What is graphql schema?
- What is a pseudo join?
- What are queries?
- What are mutations?
- What are resolvers?

If you do not understand above, the code inside `lectorial09/example2.backend/src/graphql` will not make any sense.

Below is the snippet of the two most important files `resolvers.ts` and `schema.ts`

```
/* resolvers.ts */
export const resolvers = { Query: {
  profiles: async () => {
    return await profileRepository.find();
  },
},
Mutation: {
  createProfile: async (_, args: any) => { const profile =
    profileRepository.create(args); return await
    profileRepository.save(profile);
  },
},
};
```

```
/* schema.ts */
import gql from "graphql-tag";

export const typeDefs = gql` type Pet {
  pet_id: ID! name:
  String!
  profiles: [Profile!]
}

type Profile { profile_id: ID!
  email: String! first_name:
  String!

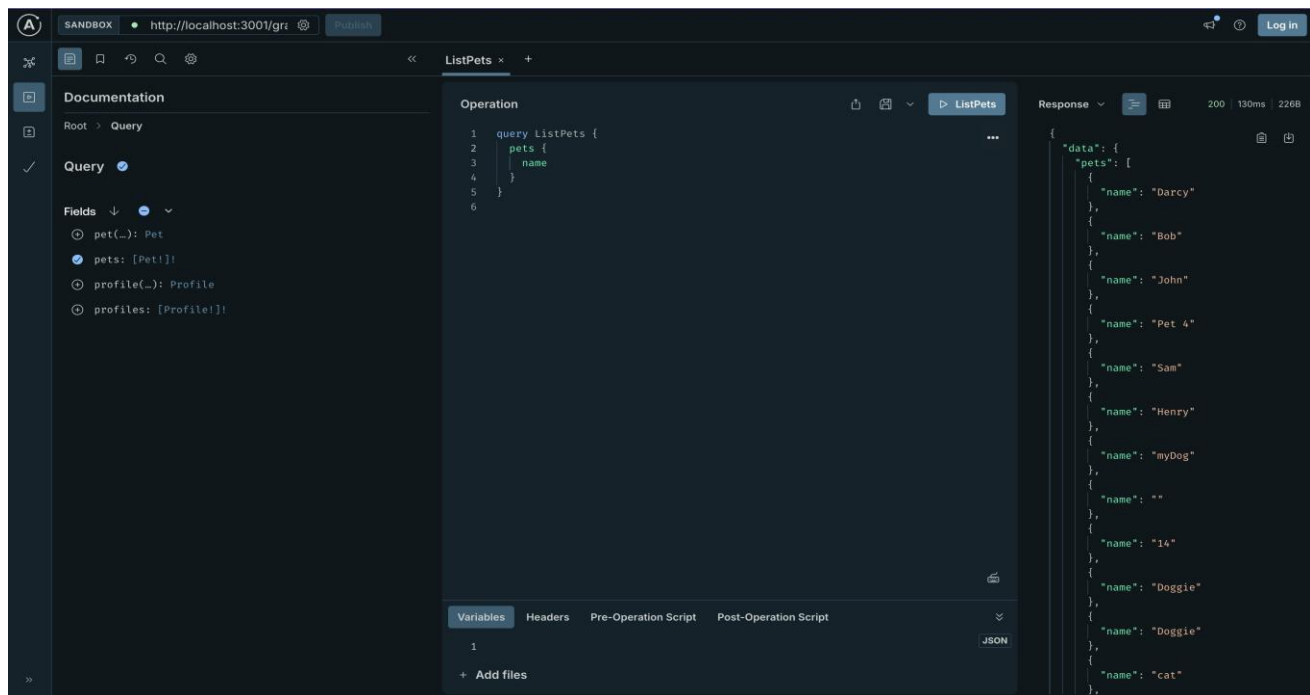
  last_name: String! mobile:
  String street: String city:
  String state: String
  postcode: String pets:
  [Pet!]
}
`;
```

GraphQL Explorer

As part of this stack and by using GraphQL we get access to an interactive debugging tool.

Ensure the project is running by running `npm run dev`

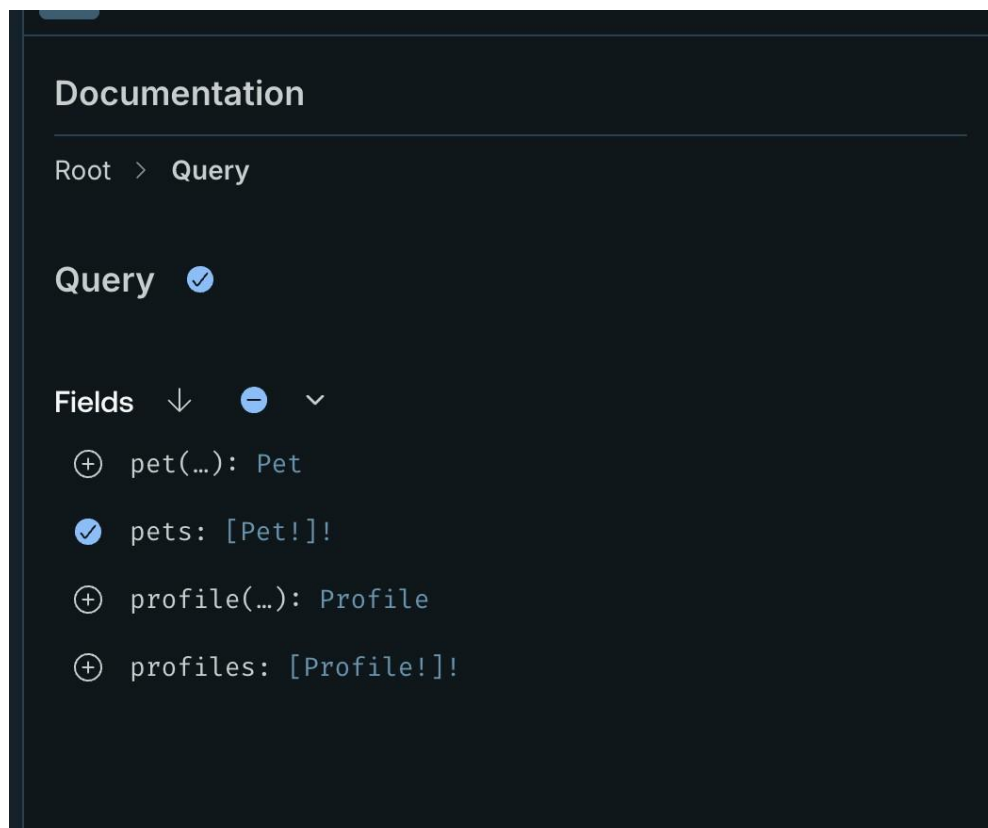
Then visit `http://localhost:3001/graphql`



From here:

- Try running some queries against your GraphQL server
- Try running some mutations against your GraphQL server

You can also explore documentation on the left-hand side of the window:



Activity 2 - Creating a frontend that connects to our GraphQL backend

In activity 2, we will create a new NextJS app that connects to our GraphQL backend.

Firstly, create a new Next app with `npxcreate-next-app`, picking all the standard options, and calling your project frontend.

Note: We do provide an example frontend codebase, but try and complete these steps yourself

Next, change directories into your project `cd frontend`

Next, we need to install some additional dependencies that allow our React frontend to connect to our GraphQL backend.

- `npm install @apollo/client graphql`

Now that we have all of our dependencies installed, let's change our app to be a basic one that lists all of the pets, feel free to extend from here to use mutations and the like.

Create a services folder

Create a new folder called `services` that sit under `src`. From here we must configure the Apollo client to connect to our backend.

Create the Apollo Client

Create a file called `apollo-client.ts` that has the following contents:

```
import { ApolloClient, InMemoryCache, createHttpLink } from "@apollo/client";

const httpLink = createHttpLink({
  uri: "http://localhost:3001/graphql", // This is where our server is listening for GraphQL requests
});

export const client = new ApolloClient({ link: httpLink,
  cache: new InMemoryCache(),
});
```

Create our api.ts file

Our api.ts file will contain both our GraphQL queries, as well as helper functions to help us call them. Paste the following into api.ts

```
import { gql } from "@apollo/client"; import { client } from
"./apollo-client";

// Our typescript interface for a pet export interface Pet {
  pet_id:string; name:
  string; profiles?: {
    profile_id:string; email: string;
    first_name:string; last_name:
    string;
  }[];
}

// GraphQL Queries const
GET_PETS=gql`
  query GetPets{ pets {
    pet_id
    name
  }
}
`;

export const petService = {
  getAllPets: async (): Promise<Pet[]> => {
    const { data } = await client.query({ query: GET_PETS }); return data.pets;
  },
};
```

Using our query inside a component.

Inside our `src/pages/index.ts` add the following code to call our service, which in turn calls our GraphQL

```
import { useState } from "react"; import { useEffect } from
"react";
import { Pet, petService } from "../services/api";

export default function Home() {
  const [pets, setPets] = useState<Pet[]>([]);

  useEffect(() => { petService.getAllPets().then(setPets);
  }, []);

  return (
    <div className="p-4">
      <h1 className="text-2xl font-bold">Pets</h1>

      <ul>
        {pets.map((pet) => (
          <li key={pet.pet_id}>
            <p>{pet.name}</p>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Finally, boot up our frontend (so that both services are running)

`npm run dev`

Activity 3

There are many developers in the industry who think:

- they are beyond code commenting and documentation
- that documentation is a waste of time
- their code is so amazing that everyone can comprehend it without any form of documentation
- that code commenting and documentation is an outdated concept
- that any kind of agile project does not need documentation

But all the above is a recipe for disaster. The above are typical examples of –unprofessional thinking. In fact, there are developers who do not document on purpose.

As per true agile manifesto and thinking

When Documentation Becomes Important:

- You have multiple teams working on the same project.
- You are building some component library or complex logic that is used by many people
- You forget things easily

Advantages of Having Good Documentation:

- It's a sign that the developers care about this project
- It helps new people to understand the project quickly
- Good documentation reduces the chance of misunderstanding
- The documentation itself can be a great asset besides your application
- Good documentation can sometimes work as a style guide for others. Documentation increases the value of a project.

There are many ways to document your code:

- Using React Styleguidist
- Using React-Docz
- Using React-docgen
- Using Storybook
- Using JSDoc
- Using Bit

Documenting using JSDoc

JSDoc also works for Typescript!

You can integrate this tool with Visual Studio Code.

Install jsdoc via extensions page of VS code – Go through these to learn how to document your react code and generate a documentation:

- JSDoc for React.js: <https://www.inkoop.io/blog/a-guide-to-js-docs-for-react-js/>
- JSDoc style guide: <https://github.com/shri/JSDoc-Style-Guide>